

サービスコール

モーション追加・歩行パラメータ調整

実機で「何を触れば動きが変わるか」を最初に確認し、
その後でコード全体を理解するための実務資料

この資料の順番

最初に「モーションを追加する」「歩行を調整する」を見る。
その後に Motion / Config / main / FootController / FootIK の詳細を読む。

対象	基準ソース	主な役割
歩行調整	Config.cpp / Motion.cpp	歩容パラメータと歩行状態機械
教示モーション	Motion.cpp	角度キーフレームと Point-to-Point 補間
足先計算	FootController.h / FootIK.h	足先 Pose から関節角を求める API と IK
実行管理	main.cpp	入力、排他制御、モーション呼び出し

対象機体: EN 班「サービスコール」 / 作成日: 2026-09-11

最初に見る - モーションを追加する

モーション追加には、大きく 2 つの作り方があります。

A. 歩容のように「計算式 + 足先軌道 + 逆運動学」で作る

向いているもの: 歩行・横移動・旋回など、周期運動を数式で再現したい場合。

考え方: 「時間 $t \rightarrow$ 足先座標 $(X, Y, Z) \rightarrow$ IK $\rightarrow$ サーボ角」。

ブラックボックス部分があっても、この流れを理解すれば調整できます。

B. 教示したポーズを「Point-to-Point」でつなぐ

向いているもの: 攻撃・起き上がり・ポーズ・演技など、決まった姿勢の連続。

考え方: 「教示姿勢 1 $\rightarrow$ 教示姿勢 2 $\rightarrow$ 教示姿勢 3」を順番につなぐ。

現在のコードでは `calculateStepMotion() + motion_sub()` がこの役割です。

A. 計算式 + 逆運動学で歩容を追加する

1. まず「1 周期の中で足がどう動くか」を決める。例: 接地 $\rightarrow$ 後ろへ進む $\rightarrow$ 遊脚で足を上げる $\rightarrow$ 前へ戻す。
2. 次に、時間 ts から足先の目標座標を作る。現在の実装では `FootController::tread(h, Wd, DutyX, DutyY, T, ts)` がその役割を持つ。
3. 左右の足に位相差を付ける。現在の歩行は右足を $T/2$ ずらして交互に動かしている。
4. 足先 Pose を作り、`FootController::setTargetPose()` に渡す。
5. `setTargetPose()` の内部で逆運動学を使い、足先座標を 5 関節の角度に変換する。
6. 最後に `Motion.cpp` の `walk` 系関数へ登録し、`main.cpp` の `taskManager()` から呼べるようにする。

覚え方

歩容は「角度を直接決める」のではなく、まず足先の動きを決める。

最後に IK が関節角へ変換する。

B. 教示による Point-to-Point モーションを追加する

1. `LOCK_DebugMode()` でサーボをフリーにして、実機を手で動かす。
2. シリアルに出る 19 個の角度をコピーし、必要なポーズを複数作る。例: [待機] $\rightarrow$ [構え] $\rightarrow$ [攻撃] $\rightarrow$ [戻り]。
3. 各ポーズ間を `calculateStepMotion()` で補間し、区間ごとの `totalSteps` で速さ・溜め・戻しを決める。
4. 最後に `Motion.h` と `main.cpp` へ登録する。

最初に見る - 歩行パラメータを変える

歩行調整で最初に触る場所は Config.cpp です。

やりたいこと	まず変更する項目	感覚的な意味
歩く速さを変えたい	T	1歩の時間。小さくすると速くなる
歩幅を変えたい	Wd	足を前後に動かす量
足を高く上げたい	h	遊脚の高さ。まず少しずつ上げる
機体を高く/低くしたい	offsetZ_left / right	脚を伸ばす/折りたたむ方向
前後に倒れそう	offsetX_left / right	足の前後位置をずらして重心を調整
左右の足幅を変えたい	Spac	左右の足の横方向の間隔
蹴りを長く使いたい	kickTime	接地終盤のキック区間の長さ

重要

Config.cpp の値を変えても、その値が Motion.cpp で係数を掛けられたり上書きされたりしていることがあります。

「設定値 = 実際の動作」とは限りません。

歩行タイプと設定場所

操作 / 関数	使う設定	現行コードの対応
通常前進: motion::walk::walk1()	Config::MV_X_PARAM_3	前後歩行
横歩行: motion::walk::walkY()	Config::MV_X_PARAM_1Y	方向ボタンによる横移動
旋回: motion::walk::turn()	Config::MV_X_PARAM_TURN	L3 押下中の旋回

現在の値を変更する場所

Config.cpp → Config::MV_X_PARAM_3 / MV_X_PARAM_1Y / MV_X_PARAM_TURN

調整するときの順番

1. まず姿勢を合わせる: offsetZ → offsetX → Spac
2. 次に歩き方を合わせる: T → Wd → h
3. その後に必要なら kickTime / kickX / kickY を確認する
4. 左右差が出たら left / right を個別に調整する
5. 最後に実機で 1 つずつ変更して、何が改善したか記録する

実例 1 - attack_GotHand_left を追加する

この章だけで追加作業を完了できる

ここでは「左腕 4 軸を教示し、待機 → 構え → GotHand → 待機へ戻る」Point-to-Point モーションを、attack_GotHand_left という名前で追加する例を最後まで追います。

実際の角度値だけは LOCK_DebugMode() で取得した教示値に置き換えます。

まず完成形を頭に入れる

作業	触るファイル	やること
① 教示	実機 + Serial	必要なポーズを 3~4 個作り、19 角度を保存する
② 宣言	Motion.h	attack_GotHand_left() を宣言する
③ 本体	Motion.cpp	ポーズ配列 + motion_sub() + taskPhase を書く
④ 呼び出し	main.cpp	空いているボタン入力に登録する
⑤ 実機確認	実機	速度・到達姿勢・干渉・戻りを確認する

Step 1 - LOCK_DebugMode() でポーズを作る

1. PS ボタンで LOCK_DebugMode() を起動する。
2. フリー状態でロボットの左腕を「待機」「GotHand の構え」「GotHand の実行姿勢」にそれぞれ動かす。
3. 各姿勢が決まった瞬間に Serial に出た { ... } の 19 個の値をコピーする。
4. 左腕だけを使う今回の例では、ServoArray[1]~[4] に対応する 4 値を取り出して使う。
5. Cross を押してデバッグモードを終了する。

ここで保存するもの

P0 = 待機姿勢 / P1 = 構え / P2 = GotHand / P3 = 戻し用姿勢。

P0 と P3 は同じでもよい。P1→P2 を短く、P2→P3 を少し長くすると「溜めてから戻る」動きにしやすい。

Step 2 - Motion.h に名前を追加する

```
namespace motion{
    namespace posture{
        namespace battle{
            bool attack_GotHand_left();
        }
    }
}
```

```
}  
}
```

既存の `attack_Light_left()` と同じ階層に置きます。Motion.h は「このモーションが存在する」と他のファイルへ知らせる場所です。

Step 3 - Motion.cpp に本体を追加する

```
bool motion::posture::battle::attack_GotHand_Left(){  
    static int taskPhase = 0;  
    static int currentStep = 0;  
  
    // LOCK_DebugMode() で取得した左腕 4 軸の値へ置き換える  
    float pose0[4] = { /* P0_J1, P0_J2, P0_J3, P0_J4 */ };  
    float pose1[4] = { /* P1_J1, P1_J2, P1_J3, P1_J4 */ };  
    float pose2[4] = { /* P2_J1, P2_J2, P2_J3, P2_J4 */ };  
  
    int index = leftArm_index_num;  
  
    switch(taskPhase){  
        case 0:  
            currentStep = 0;  
            taskPhase = 1;  
            break;  
  
        case 1:  
            motion_sub(15, index, 4, currentStep, taskPhase, pose0, pose1);  
            break;  
  
        case 2:  
            motion_sub(8, index, 4, currentStep, taskPhase, pose1, pose2);  
            break;  
  
        case 3:  
            motion_sub(25, index, 4, currentStep, taskPhase, pose2, pose0);  
            break;  
  
        case 4:  
            taskPhase = 0;  
            currentStep = 0;  
            return false;  
    }  
  
    return true;  
}
```

読み方

15 / 8 / 25 は「区間の長さ」。値を大きくすると、その区間をゆっくり動かします。
`motion_sub()` が毎フレーム 1 段階進み、終了すると `taskPhase` を次へ進めます。

Step 4 - activeMotion に登録する

ここが GotHand 追加で見落とししやすい場所です。activeMotion は「そのモーションが現在実行中か」を保持するフラグ群です。main.cpp の namespace activeMotion::posture::battle に、GotHand 用の bool を 1 個追加します。

```
namespace activeMotion{
    namespace posture{
        namespace battle{
            bool attack_GotHand_Left;
        }
    }
}
```

必ず 3 か所をそろえる

1. Motion.h の関数宣言
 2. Motion.cpp の関数本体
 3. main.cpp の activeMotion フラグ + runExclusiveTask() 登録
- この 3 つの名前が一致していることを確認します。

Step 5 - main.cpp から呼べるようにする

```
runExclusiveTask(
    button_bits == SQUARE_BIT, // 例。実際には空いている入力へ割り当てる
    LEFT_ARM_BIT,
    activeMotion::posture::battle::attack_GotHand_Left,
    motion::posture::battle::attack_GotHand_Left
);
```

ここで重要なこと	理由
LEFT_ARM_BIT を指定する	左腕 4 軸だけをモーションの占有対象にできる
button_bits の条件を決める	既存の攻撃モーションと入力が競合しないようにする
activeMotion の同じフラグを渡す	runExclusiveTask() が「実行中か」を保持する

Step 6 - 実機確認はこの順番

1. まず「P0 → P1」だけ確認する。腕が安全な方向へ動くかを見る。
2. 次に「P1 → P2」。肩・肘・手先が他の部位へ干渉しないかを見る。
3. 最後に「P2 → P0」。戻りで勢いが出ないかを見る。
4. 動きが急なら totalSteps を増やす。動きが遅すぎるなら減らす。
5. 途中姿勢を修正したい場合は、P1 と P2 の間に P1.5 を追加して別区間にする。

実例 2 - walkY を「もう少し落ち着いた動き」にする

最初に触るのは Config.cpp

横歩行は Config::MV_X_PARAM_1Y を使っています。まず Config 側だけを少し穏やかにし、それでも急なら Motion.cpp のハードコードを確認します。

現在の walkY 設定

```
GaitParameters Config::MV_X_PARAM_1Y{
    0.5,    // T
    50,     // h
    10,     // Wd
    0.90,   // DutyX
    0.85,   // DutyY
    30,     // Fps
    25,     // Spac
    307.0f, // offsetZ_left
    -18.0f, // offsetX_left
    300.0f, // offsetZ_right
    -18.0f, // offsetX_right
    -0.0f,  // kickX_left
    -10.0f, // kickY_left
    -0.0f,  // kickX_right
    -10.0f, // kickY_right
    0.2f    // kickTime
};
```

変更例 - まずは「ゆっくり・小さく」

```
GaitParameters Config::MV_X_PARAM_1Y{
    0.7,    // T    0.5 -> 0.7 : 1周期を長くする
    45,     // h    50 -> 45 : 足上げを少し控えめ
    8,      // Wd   10 -> 8  : 横移動量を少し小さく
    0.90,   // DutyX
    0.85,   // DutyY
    30,     // Fps
    25,     // Spac
    307.0f, // offsetZ_left
    -18.0f, // offsetX_left
    300.0f, // offsetZ_right
    -18.0f, // offsetX_right
    -0.0f,  // kickX_left
    -10.0f, // kickY_left
    -0.0f,  // kickX_right
    -10.0f, // kickY_right
    0.2f    // kickTime
};
```

この数値は「調整開始点」の例

機体の重量・床・脚長などが違えば最適値は変わります。この例の意図は「Tを長く」「Wdを少し小さく」「hを少し小さく」して、変化量を一度に大きくしないことです。

なぜ T を大きくすると落ち着きやすいのか

変更	期待する変化	見るポイント
T 0.5 → 0.7	1 周期が長くなる	足の切替が急でなくなるか
Wd 10 → 8	横方向の移動量が小さくなる	左右への振り出しが減るか
h 50 → 45	遊脚の上下動を少し減らす	足先が跳ねる感じが減るか

それでも急なら Motion.cpp を見る

walkY() には Config にはない固定値があります。特に次の3つは、ボタン量に対して横幅補正・Y方向補正・足首 Roll 補正を直接作っています。したがって「Config だけでは落ち着かない」ことがあります。

```
float space_def =15 * buton_lx / 100.0;  
float offsetY_def =20 * buton_lx / 100.0;  
float angle_def = (5*PI/360.0)*buton_lx / 100.0;
```

調整の順番

① まず Config の T / Wd / h を変更 → ② まだ横へ振るなら space_def / offsetY_def を確認 → ③ 足首の傾きが大きいなら angle_def を確認。

いきなり全部を変えないことが重要です。

walkY 調整の実験メモ

試行	T	Wd	h	結果
初期	0.5	10	50	
1 回目	0.7	8	45	
2 回目				
3 回目				

ここまでの章だけでできること

- 新しい左腕モーションを 1 本追加できる。
- 教示データをどこへ入れるか分かる。
- モーションをどのボタンへ登録するか分かる。
- walkY の調整対象が Config::MV_X_PARAM_1Y だと即座に分かる。
- 横歩行が急なときに Config と Motion.cpp のどちらを見るべきか判断できる。

実例 3 - 「うまく動かない」 ときの切り分け表

症状	最初に見る場所	次に見る場所	対処の方向
モーションが呼ばれない	main.cpp のボタン条件	busyPartsBit	入力条件と占有状態を確認
モーション開始直後に終わる	Motion.cpp の taskPhase	motion_sub() の戻り値	phase 遷移を確認
腕の一部だけ動かない	ServoArray index	leftArm_index_num	対象サーボの範囲を確認
歩行設定を変えても変化が小さい	Motion.cpp の係数・固定値	Config.cpp	上書きや *0.0f を探す
歩幅は変わるが足がぶつかる	h / offsetZ / Spac	IK / Pose	姿勢と可動範囲を確認
横歩行が急に見える	T / Wd	space_def / offsetY_def	大きな固定補正を疑う

困ったら「どこで値が変わったか」を追う

Config の値 → Motion.cpp のローカル変数 → tread() の入力 → Pose → setTargetPose() → IK → サーボ角、の順で見えていけば、ブラックボックスがあっても切り分けできます。

1. モーション追加の詳しい方法

1.1 歩容方式 - 計算式と逆運動学

歩行系は、関節角を1個ずつ教示して再生する方式ではなく、足先の軌道を計算し、そこから関節角を逆運動学で求める方式です。FootController.h には Pose、tread()、setTargetPose() など、歩容を構成するための API が定義されています。

層	何を決めるか	現行コード
歩行モーション	時間・入力・左右位相	Motion.cpp の walk1 / walkY / turn
軌道生成	足先 X/Y の周期軌道	FootController::tread()
足先 Pose	X/Y/Z/Roll/Pitch/Yaw	FootController::Pose
逆運動学	5 関節角への変換	FootController::setTargetPose() / IK
サーボ指令	角度を実機へ送る	servoCS::Servo

実際に新しい歩容を作るとき

「足先の軌道式を変える」なら FootController 側も対象になります。今回提供されている FootController.h には API はありますが、tread() の実装本体はありません。そのため、tread() の内部数式まではこの資料では断定しません。

1.2 教示方式 - Point-to-Point

現在の getUp() や攻撃モーションは、複数の角度配列をキーフレームとして持ち、motion_sub() で区間ごとに進める方式です。calculateStepMotion() は smoothstep によって、開始と終了で速度が0になるよう補間します。

```
float calculateStepMotion(float start_angle, float target_angle,
                        int current_step, int total_steps) {
    if (current_step >= total_steps) return target_angle;
    float t = (float)current_step / (float)total_steps;
    float smooth_t = t * t * (3.0f - 2.0f * t);
    return start_angle + (target_angle - start_angle) * smooth_t;
}
```

1.3 教示モーションの実作業手順

1. PS ボタンで LOCK_DebugMode() を起動する。
2. Circle でフリー/再生を切り替えながら、Cross を押すまで教示を行う。
3. フリーモードでは 19 個のサーボを setPosFree() にし、その時点の getPosDeg() をシリアルへ出力する。
4. シリアルに出た {角度 1, ..., 角度 19} をポーズデータとして保存する。
5. 動作を「待機→構え→動作→戻り」のように区切り、各ポーズをキーフレームへする。

6. 各区間に totalSteps を設定し、motion_sub() を呼ぶ。

注意

LOCK_DebugMode() は while と delay() を含むため、通常のノンブロッキング設計とは別扱いです。教示時だけ使うデバッグ用機能として理解してください。

2. 歩行関数と Config のつながり

歩行関数	参照する設定	主な動作
motion::walk::walk1()	Config::MV_X_PARAM_3	通常前進。左スティックで左右の Wd 等を調整
motion::walk::walkY()	Config::MV_X_PARAM_1Y	方向ボタンで横方向の動作
motion::walk::turn()	Config::MV_X_PARAM_TURN	左右どちらかを遊脚側にして旋回

walk1() / walkY() / turn() はいずれも Config の GaitParameters を参照し、毎フレーム足先 Pose を作って setTargetPose() へ渡します。walk1() では startWalking() → 通常歩行 → endWalking() の状態遷移を持ちます。

3. GaitParameters の全項目

項目	意味	調整の見方	注意点
T	周期 [s]	小さくすると速く、大きくすると遅くなる	startWalking/endWalking では係数が入る
h	足上げ高さ	つまずきがあるとき上げる	start/end では 0.5 倍
Wd	接地時の前後方向の軌道量	歩幅・進み方を変える	walk1 では stick_lx に応じて左右へ補正
DutyX	接地/遊脚の切替比率に関する値	歩容の位相を変える	Motion.cpp では support_end 計算と tread() 引数に使用
DutyY	横方向のデューティ比	tread()へ渡される	内部実装未提供のため最終的な効果は断定不可
Fps	歩行軌道を何分割するか	変更は滑らかさと計算負荷に影響	1 周期の totalFrames = T * Fps
Spac	左右の足幅	足を広げる/狭める	左右 Pose の Y 方向に使われる
offsetZ_left/right	左右足の高さ基準	大きいほど脚を伸ばす側、小さいほど折りたたむ側	Pose 式の符号に注意
offsetX_left/right	左右足の前後基準	前後に倒れそうとき重心調整	startWalking では -15.0f に上書き
kickX_left/right	キック距離 X	後ろ方向などの押し込み量	現行歩行では *0.0f で無効化
kickY_left/right	キック距離 Y	下方向などの押し込み量	現行歩行では *0.0f で無効化
kickTime	キック時間 [s]	接地終盤のキック区間を長く/短くする	kick 値がゼロ化されている間は見た目に効かない

4. 現行コードの「Config を変えても動きが変わらない」箇所

場所	現行コード	意味
startWalking()	T = param_p.T * 0.90f	開始時の周期だけ短縮
startWalking()	h = param_p.h * 0.5f	開始時の足上げを半分
startWalking()	offsetX_left/right = -15.0f	Config 値を使わず固定値
startWalking() / endWalking() / walk1() / walkY() / turn()	kickX/kickY * 0.0f	Config のキック値を実質無効化
endWalking()	T = param_p.T * 0.70f	停止移行時だけ周期短縮

調整の原則

Config.cpp は「元の設定値」。Motion.cpp は「その値を実際にどう使うか」。
両方を見るのが最短です。

5. パーツ単位の排他制御

main.cpp では busyPartsBit と runExclusiveTask() を使い、腰・左腕・右腕・左足・右足をビットで管理しています。歩行は左右足の2ビットを占有し、左腕攻撃は左腕だけを占有するため、歩行しながら左腕を動かすことができます。

ビット	対象	例
HIP_BIT	腰	hip()
LEFT_ARM_BIT	左腕	attack_Light_left()
RIGHT_ARM_BIT	右腕	attack_Light_right()
LEFT_FOOT_BIT	左足	walk 系
RIGHT_FOOT_BIT	右足	walk 系

```
runExclusiveTask(
    startFlag,
    LEFT_ARM_BIT,
    activeMotion::posture::battle::attack_Light_left,
    motion::posture::battle::attack_Light_left
);
```

6. ファイルごとの役割

ファイル	役割	触る場面
Motion.h	モーション関数の宣言	新しいモーション名を公開するとき
Motion.cpp	モーション本体・状態機械・補間・歩容	動きを作るとき
Config(1).h	GaitParameters の項目定義・Config 宣言	新しい設定項目を増やすとき
Config.cpp	実際の歩行プリセット値	歩行の数値を調整するとき
main.cpp	入力とモーション実行管理	ボタン登録・排他条件を変えるとき
FootController.h	足の API・Pose・tread・IK 呼出し口	足先軌道や姿勢指定の設計を確認するとき
FootIK.h	足先 Pose → 関節角の逆運動学	IK 式を変更するとき
Declaration.h	グローバルな Servo/FootController 等の宣言	共有オブジェクトを追加するとき
AssistiveProgram.*	補助関数・FrameLimiter・LittleFS 等	共通処理を追加するとき

7. ノンブロッキング・状態機械の読み方

通常のモーションは「一度呼んで最後まで動く関数」ではなく、「1 フレーム進んだところで戻る関数」です。taskPhase と currentStep を保持し、次の loop で続きを実行します。終了したら false を返し、runExclusiveTask() が busyPartsBit を解放します。

戻り値	意味
true	まだ実行中。次のフレームでも呼ぶ
false	そのモーションは終了。占有していたパーツを解放する

8. 実機での調整手順

1. 安全な初期姿勢を作る。
2. LOCK_DebugMode() で各関節の基準角度を確認する。
3. Config の値を 1 つだけ変更する。
4. startWalking/endWalking/歩行本体で上書きされていないか確認する。
5. 必要なら Pose の値までログを入れ、Config → Pose → IK のどこで差が出るか切り分ける。
6. 実機では T → Wd → h → offsetZ → offsetX → Spac の順に大枠から調整する。
7. 最後に kickTime / kickX / kickY を確認する。現状はキック値がゼロ化されているため、まず有効化の有無を確認する。

9. コピペ用テンプレート

9.1 教示 Point-to-Point モーション

```
namespace motion {
    namespace posture {
        bool myMotion();
    }
}

bool motion::posture::myMotion(){
    static int taskPhase = 0;
    static int currentStep = 0;

    float pose0[4] = { /* 初期姿勢 */ };
    float pose1[4] = { /* 教示姿勢 1 */ };
    float pose2[4] = { /* 教示姿勢 2 */ };
    int index = leftArm_index_num;

    switch(taskPhase){
        case 0:
            currentStep = 0;
            taskPhase = 1;
            break;
        case 1:
            motion_sub(15, index, 4, currentStep, taskPhase, pose0, pose1);
            break;
        case 2:
            motion_sub(10, index, 4, currentStep, taskPhase, pose1, pose2);
            break;
        case 3:
            motion_sub(20, index, 4, currentStep, taskPhase, pose2, pose0);
            break;
        case 4:
            taskPhase = 0;
            currentStep = 0;
            return false;
    }
    return true;
}
```

9.2 Motion.h へ宣言

```
namespace motion{
    namespace posture{
        bool myMotion();
    }
}
```

9.3 main.cpp へ登録

```
runExclusiveTask(
    button_bits == TRIANGLE_BIT,
```

```

LEFT_ARM_BIT,
activeMotion::posture::myMotion,
motion::posture::myMotion
);

```

9.4 Config.cpp の調整

```

GaitParameters Config::MV_X_PARAM_3{
    0.9,    // T
    50,     // h
    80,     // Wd
    0.95,   // DutyX
    0.85,   // DutyY
    30,     // Fps
    30,     // Spac
    312.0f, // offsetZ_left
    -20.0f, // offsetX_left
    310.0f, // offsetZ_right
    -20.0f, // offsetX_right
    -0.0f,  // kickX_left
    -1.0f,  // kickY_left
    -0.0f,  // kickX_right
    -1.0f,  // kickY_right
    0.5f    // kickTime
};

```

9.5 activeMotion + runExclusiveTask テンプレート

```

namespace activeMotion{
    namespace posture{
        namespace battle{
            bool myMotion;
        }
    }
}

runExclusiveTask(
    button_bits == TRIANGLE_BIT,
    LEFT_ARM_BIT,
    activeMotion::posture::battle::myMotion,
    motion::posture::battle::myMotion
);

```

9.6 2点だけの最小モーション

```

bool motion::posture::battle::myMotion(){
    static int taskPhase = 0;
    static int currentStep = 0;

    float pose0[4] = { /* 待機 */ };
    float pose1[4] = { /* 動作 */ };
    int index = leftArm_index_num;

    switch(taskPhase){
        case 0:

```

```

        currentStep = 0;
        taskPhase = 1;
        break;
    case 1:
        motion_sub(20, index, 4, currentStep, taskPhase, pose0, pose1);
        break;
    case 2:
        motion_sub(30, index, 4, currentStep, taskPhase, pose1, pose0);
        break;
    case 3:
        taskPhase = 0;
        currentStep = 0;
        return false;
    }
    return true;
}

```

9.7 3点の「溜め → 本動作 → 戻し」テンプレート

```

// P0: 待機
// P1: 構え
// P2: 実行姿勢

motion_sub(25, index, 4, currentStep, taskPhase, pose0, pose1);
// 構えをゆっくり作る

motion_sub(8, index, 4, currentStep, taskPhase, pose1, pose2);
// 短い区間は本動作を速く見せやすい

motion_sub(35, index, 4, currentStep, taskPhase, pose2, pose0);
// 戻しを長くすると終端を落ち着かせやすい

```

9.8 walkY 調整メモの書き方テンプレート

```

// 変更前
T   = 0.50
Wd  = 10
h   = 50

// 変更後（例）
T   = 0.70 // 1周期を長くする
Wd  = 8    // 移動量を少し小さくする
h   = 45   // 足上げを少し控える

// 実機メモ
// ・横への振り出し: 減った / 変わらない / 増えた
// ・足の引っ掛かり: 改善 / 同等 / 悪化
// ・停止時のふらつき: 改善 / 同等 / 悪化

```

10. 客観的な設計評価・改善提案

現行構成は、競技ロボットの開発として「実機で素早く試す」点では強みがあります。一方で、設定値とアルゴリズム側の補正值が Motion.cpp に混在しているため、複数人で調整すると「なぜこの数値を変えたのか」が追いにくくなります。

論点	現状	改善案
歩行パラメータ	Config にまとまっているが Motion 側で係数・上書きあり	Runtime 補正を別構造体へ集約
キック	設定値が *0.0f で無効化されている	ENABLE_KICK 等を明示して意図をコードにする
座標系	kickX/Y などの意味が読み取りにくい	足先座標系を 1 枚図にして命名を統一
モーション登録	Motion.h / main.cpp / Motion.cpp に分散	追加時のチェックリスト化
調整の再現性	変更値と結果の記録が別管理	実験表に T/Wd/h/offset/結果をセットで残す

率直な評価

最優先の改善は「Config に書いた値が最終的にどこでどう変換されるか」を追えるようにすることです。ここが整理されれば、歩行調整の引き継ぎコストが大きく下がります。

付録 - 現行コードから確認できる事実

- Motion.h には walk1 / walkY / turn、攻撃、taunt、nop、LOCK_DebugMode、pose、chair、kneeling、getUp、hip が宣言されている。
- Config(1).h の GaitParameters は T / h / Wd / DutyX / DutyY / Fps / Spac / offsetZ_left / offsetX_left / offsetZ_right / offsetX_right / kickX_left / kickY_left / kickX_right / kickY_right / kickTime の順。
- main.cpp の busyPartsBit は腰・左腕・右腕・左足・右足をビットで管理している。
- LOCK_DebugMode() は 19 サーボを free にし、角度を取得して Serial に出力する。
- FootController.h には tread()、setTargetPose()、Pose、IK などが宣言されている。
- FootIK.h には Pose から 5 関節角を計算する逆運動学関数がある。

調整記録シート

項目	変更前	変更後	実機で見た結果
T			
Wd			
h			
offsetZ_left / right			
offsetX_left / right			
Spac			
kickTime / kickX / kickY			
総合評価			

記録のコツ

1 回の実験で 1 項目だけ変える。

「速くなった」だけでなく、足の接地、ふらつき、転倒方向、床との接触も一緒に残すと、次の調整が速くなります。

この資料は「まず使う → その後で理解する」の順で読むための構成です。